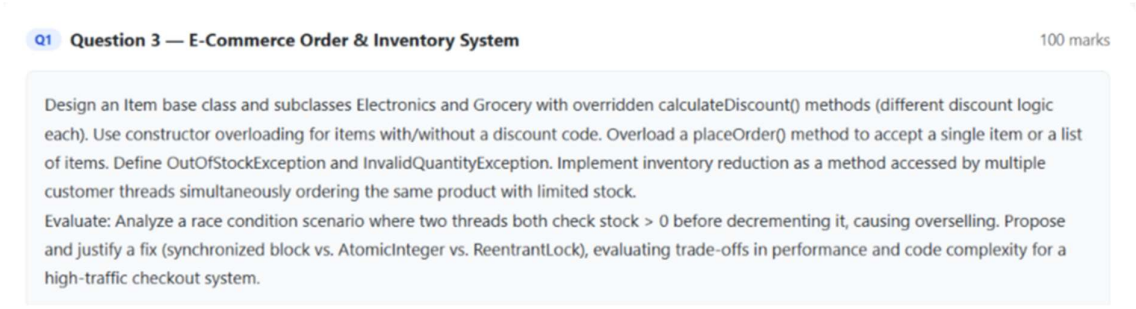
CO5 – AT2

Name: Mandati Sharan Mahi Reddy

Reg No: 192324082



Q1 **Question 3 — E-Commerce Order & Inventory System** 100 marks

Design an Item base class and subclasses Electronics and Grocery with overridden calculateDiscount() methods (different discount logic each). Use constructor overloading for items with/without a discount code. Overload a placeOrder() method to accept a single item or a list of items. Define OutOfStockException and InvalidQuantityException. Implement inventory reduction as a method accessed by multiple customer threads simultaneously ordering the same product with limited stock.
Evaluate: Analyze a race condition scenario where two threads both check stock > 0 before decrementing it, causing overselling. Propose and justify a fix (synchronized block vs. AtomicInteger vs. ReentrantLock), evaluating trade-offs in performance and code complexity for a high-traffic checkout system.

Code:

```
import java.util.*;
public class Main {
   static class OutOfStockException extends Exception {
      OutOfStockException(String message) {
         super(message);
      }
   }
   static class InvalidQuantityException extends Exception {
      InvalidQuantityException(String message) {
         super(message);
      }
   }
   static abstract class Item {
      String name;
      double price;
      String code;

      Item(String name, double price) {
         this.name = name;
```

```
        this.price = price;
    }

    Item(String name, double price, String code) {
        this.name = name;
        this.price = price;
        this.code = code;
    }

    abstract double calculateDiscount();
}

static class Electronics extends Item {
    Electronics(String name, double price) {
        super(name, price);
    }

    Electronics(String name, double price, String code) {
        super(name, price, code);
    }

    double calculateDiscount() {
        return "ELEC10".equals(code) ? price * 0.10 : 0;
    }
}

static class Grocery extends Item {
    Grocery(String name, double price) {
        super(name, price);
    }
```

```
    Grocery(String name, double price, String code) {
        super(name, price, code);
    }

    double calculateDiscount() {
        return "GROC5".equals(code) ? price * 0.05 : 0;
    }
}

static class Inventory {
    int stock;

    Inventory(int stock) {
        this.stock = stock;
    }

    synchronized void reduceStock(int quantity)
            throws OutOfStockException, InvalidQuantityException {

        if (quantity <= 0)
            throw new InvalidQuantityException("Invalid quantity");

        if (quantity > stock)
            throw new OutOfStockException("Out of stock");

        stock -= quantity;

        System.out.println(Thread.currentThread().getName()
                + " bought " + quantity
                + ", Stock = " + stock);
    }
```

```
}

static class Order {
    Inventory inventory;

    Order(Inventory inventory) {
        this.inventory = inventory;
    }

    void placeOrder(Item item, int quantity) {
        try {
            inventory.reduceStock(quantity);

            double total =
                    (item.price - item.calculateDiscount()) * quantity;

            System.out.println(Thread.currentThread().getName()
                    + " ordered " + item.name
                    + ", Total = " + total);

        } catch (Exception e) {
            System.out.println(Thread.currentThread().getName()
                    + " : " + e.getMessage());
        }
    }

    void placeOrder(Item item) {
        placeOrder(item, 1);
    }
}
```

```
public static void main(String[] args) {

    Electronics laptop =
            new Electronics("Laptop", 50000, "ELEC10");

    Grocery rice =
            new Grocery("Rice", 1000, "GROC5");

    System.out.println("Laptop Discount = "
            + laptop.calculateDiscount());

    System.out.println("Rice Discount = "
            + rice.calculateDiscount());

    Inventory inventory = new Inventory(5);
    Order order = new Order(inventory);

    order.placeOrder(rice);

    Thread customer1 = new Thread(
            () -> order.placeOrder(laptop, 3),
            "Customer-1");

    Thread customer2 = new Thread(
            () -> order.placeOrder(laptop, 3),
            "Customer-2");

    customer1.start();
    customer2.start();

    try {
```

```
            customer1.join();
            customer2.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        System.out.println("Final Stock = " + inventory.stock);
    }
}
```

Output:

```
Laptop Discount = 5000.0
Rice Discount = 50.0
main bought 1, Stock = 4
main ordered Rice, Total = 950.0
Customer-1 bought 3, Stock = 1
Customer-1 ordered Laptop, Total = 135000.0
Customer-2 : Out of stock
Final Stock = 1
```